

HDOC Day-18

Write a menu driven C-Program using switch-cases to perform digit-based operations on a positive integer. Define one separate user-defined function for each menu option, with void return type and with a simple function call (without call by value/reference). Each function should read the required input, process it, and display the result. Prepare the algorithm and test case table. Then write, compile, execute, and test the C program against the prepared use cases.

- Check whether the number is a **Strong Number**.
- Check whether the number is an **Armstrong Number**.
- Check whether the number is a **Harshad Number**.
- Check whether the number is a **Duck Number**.

Algorithm:

- **Main Algorithm**
- **Step 1:** Start
- **Step 2:** Display the menu containing four operations
- **Step 3:** Ask the user to enter their choice
- **Step 4:** Use switch-case to select the required operation
- **Step 5:**
 - If choice = 1, call checkStrong()
 - If choice = 2, call checkArmstrong()
 - If choice = 3, call checkHarshad()
 - If choice = 4, call checkDuck()
- **Step 6:** Display the result
- **Step 7:** If the choice is invalid, display "Invalid choice"
- **Step 8:** Stop

Algorithm for Strong Number:

- **Step 1:** Start
- **Step 2:** Read the positive integer
- **Step 3:** Store the original number
- **Step 4:** Extract each digit using:
 - $digit = n \% 10$
- **Step 5:** Find the factorial of each digit
- **Step 6:** Add all factorials
- **Step 7:** Compare the sum with the original number
- **Step 8:** If both are equal, display **Strong Number**; otherwise display **Not a Strong Number**
- Step 9: Stop

Algorithm for Armstrong Number:

- **Step 1:** Start
- **Step 2:** Read the positive integer
- **Step 3:** Count the number of digits
- **Step 4:** Extract each digit
- **Step 5:** Raise each digit to the power of the total number of digits
- **Step 6:** Add all the calculated values
- **Step 7:** Compare the sum with the original number
- **Step 8:** Display the result
- Step 9: Stop

• Algorithm for Harshad Number:

- **Step 1:** Start

- **Step 2:** Read the positive integer
- **Step 3:** Extract each digit
- **Step 4:** Calculate the sum of all digits
- **Step 5:** Check whether the original number is divisible by the digit sum
- **Step 6:** If divisible, display **Harshad Number**
- Step 7: Otherwise, display Not a Harshad Number
- Step 8: Stop

• Algorithm for Duck Number:

Step 1: Start

Step 2: Read the positive integer

Step 3: Extract each digit

Step 4: Check whether any digit is 0

Step 5: If zero is found, display Duck Number

Step 6: Otherwise, display Not a Duck Number

Step 7: Stop

C Program:

- `#include <stdio.h>`
- `/* Function to find factorial */`
- `int factorial(int n)`
- `{`
- `int fact = 1;`
-
- `for (int i = 1; i <= n; i++)`
- `{`
- `fact = fact * i;`
- `}`
-
- `return fact;`
- `}`
-

- /* Function to check Strong Number */
- void checkStrong()
- {
- int n, original, digit, sum = 0;
-
- printf("Enter a positive integer: ");
- scanf("%d", &n);
-
- original = n;
-
- while (n > 0)
- {
- digit = n % 10;
- sum = sum + factorial(digit);
- n = n / 10;
- }
-
- if (sum == original)
- printf("%d is a Strong number.\n", original);
- else
- printf("%d is not a Strong number.\n", original);
- }
-
- /* Function to check Armstrong Number */
- void checkArmstrong()
- {
- int n, original, temp, digit;
- int digits = 0, sum = 0;
-
- printf("Enter a positive integer: ");
- scanf("%d", &n);
-
- original = n;
- temp = n;
-
- /* Count digits */
- while (temp > 0)
- {
- digits++;

- temp = temp / 10;
- }
-
- temp = n;
-
- /* Calculate Armstrong sum */
- while (temp > 0)
- {
- int power = 1;
-
- digit = temp % 10;
-
- for (int i = 1; i <= digits; i++)
- {
- power = power * digit;
- }
-
- sum = sum + power;
- temp = temp / 10;
- }
-
- if (sum == original)
- printf("%d is an Armstrong number.\n", original);
- else
- printf("%d is not an Armstrong number.\n", original);
- }
-
- /* Function to check Harshad Number */
- void checkHarshad()
- {
- int n, original, digit, sum = 0;
-
- printf("Enter a positive integer: ");
- scanf("%d", &n);
-
- original = n;
-
- while (n > 0)
- {

- digit = n % 10;
- sum = sum + digit;
- n = n / 10;
- }
-
- if (sum != 0 && original % sum == 0)
- printf("%d is a Harshad number.\n", original);
- else
- printf("%d is not a Harshad number.\n", original);
- }
-
- /* Function to check Duck Number */
- void checkDuck()
- {
- int n, original, digit;
- int hasZero = 0;
-
- printf("Enter a positive integer: ");
- scanf("%d", &n);
-
- original = n;
-
- while (n > 0)
- {
- digit = n % 10;
-
- if (digit == 0)
- {
- hasZero = 1;
- break;
- }
-
- n = n / 10;
- }
-
- if (hasZero)
- printf("%d is a Duck number.\n", original);
- else
- printf("%d is not a Duck number.\n", original);

- }
-
- /* Main function */
- int main()
- {
- int choice;
-
- printf("\n===== NUMBER OPERATIONS MENU =====\n");
- printf("1. Check Strong Number\n");
- printf("2. Check Armstrong Number\n");
- printf("3. Check Harshad Number\n");
- printf("4. Check Duck Number\n");
- printf("Enter your choice: ");
- scanf("%d", &choice);
-
- switch (choice)
- {
- case 1:
- checkStrong();
- break;
-
- case 2:
- checkArmstrong();
- break;
-
- case 3:
- checkHarshad();
- break;
-
- case 4:
- checkDuck();
- break;
-
- default:
- printf("Invalid choice!\n");
- }
-
- return 0;
- }

Test Case Table:

Test Case	Choice	Input	Expected Output
1	1	145	145 is not a strong number
2	1	123	123 is not a strong number
3	2	153	153 is an Armstrong number
4	2	123	123 is not an armstrong number
5	3	18	18 is a Harshad number
6	3	19	19 is not a Harshad number
7	4	105	105 is a duck number
8	4	123	123 is not a duck number
9	5	-	Invalid choice

Compilation and Execution:

The following program has been compiled and executed successfully.

Test Case 1:

```
————— NUMBER OPERATIONS MENU —————
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 1
Enter a positive integer: 145
145 is not a Strong number.
```

Test Case 2:


```
(kali㉿kali)-[~]  
$ ./a.out  
  
————— NUMBER OPERATIONS MENU —————  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 1  
Enter a positive integer: 123  
123 is not a Strong number.
```

Test Case 3:

```
(kali㉿kali)-[~]  
$ ./a.out  
  
————— NUMBER OPERATIONS MENU —————  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 2  
Enter a positive integer: 153  
153 is an Armstrong number.
```

Test Case 4:

```
(kali㉿kali)-[~]  
$ ./a.out  
  
————— NUMBER OPERATIONS MENU —————  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 2  
Enter a positive integer: 123  
123 is not an Armstrong number.
```

Test Case 5:

```
(kali㉿kali)-[~]  
$ ./a.out  
  
————— NUMBER OPERATIONS MENU —————  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 3  
Enter a positive integer: 18  
18 is a Harshad number.
```

Test Case 6:

```
(kali㉿kali)-[~]  
$ ./a.out  
  
————— NUMBER OPERATIONS MENU —————  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 3  
Enter a positive integer: 19  
19 is not a Harshad number.
```

Test Case 7:

```
(kali㉿kali)-[~]  
$ ./a.out  
  
————— NUMBER OPERATIONS MENU —————  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 4  
Enter a positive integer: 105  
105 is a Duck number.
```

Test Case 8:

```
(kali㉿kali)-[~]  
$ ./a.out  
  
————— NUMBER OPERATIONS MENU —————  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 4  
Enter a positive integer: 123  
123 is not a Duck number.
```

Test Case 9:

```
(kali㉿kali)-[~]  
$ ./a.out  
  
————— NUMBER OPERATIONS MENU —————  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 5  
Invalid choice!
```